

CORE CONCEPTS

- **Supervised:** labels. Regression (number) / classification (category). **Unsupervised:** structure without labels (clustering, dimensionality reduction). **Reinforcement:** reward from actions.
- Goal: **generalise** to unseen data. Split: train (fit), validation (tune), test (touch once for the final score).
- **Bias** = error from too simple a model (underfit). **Variance** = sensitivity to the training sample (overfit). Total error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$.

Symptom	Diagnosis	Fix
train bad, val bad	high bias	richer model, features, less regularisation
train good, val bad	high variance	more data, regularise, simpler, bagging, early stop
both good, prod bad	leakage / drift / skew	audit features, monitor

- **Learning curve:** error vs training size. Curves that converge high: bias. A persistent gap: variance (more data helps).
- Parametric (fixed number of weights: linear, logistic) vs non-parametric (grows with data: KNN, trees).
- Loss: MSE (regression, outlier-sensitive), MAE (robust), Huber, log loss / cross-entropy (classification), hinge (SVM).

REGULARISATION

- Add a penalty on weights: **L2 (ridge)** $\lambda \sum w^2$ shrinks smoothly; **L1 (lasso)** $\lambda \sum |w|$ sets some weights exactly to 0 (feature selection); **elastic net** mixes both (handles correlated features).
- Why L1 is sparse: the diamond-shaped constraint has corners on the axes; L1's pull is constant, L2's pull fades as $w \rightarrow 0$.
- Larger λ = simpler model (more bias, less variance). Choose λ by cross-validation. **Scale features first.**
- Also: dropout, early stopping, max depth, pruning, data augmentation, more data.

DATA LEAKAGE

Model sees information at training that will not exist at prediction time. Scores look great, production fails.

- **Target leakage:** a feature derived from the outcome ("refund issued" predicting fraud; "days since account closed" predicting churn).
- **Train-test contamination:** scaling, imputing, encoding or feature selection fitted on all data; duplicates across the split; SMOTE before splitting.
- **Temporal:** random split on time-ordered data (use forward chaining); features computed with future data.
- **Group:** same customer or patient in train and test (use group k-fold).
- Fix: split first, fit every step in a Pipeline on training folds only; ask "will this exist at prediction time?"; distrust near-perfect scores.

LINEAR REGRESSION

- $y = Xw + \epsilon$, fit by least squares: $w = (X^T X)^{-1} X^T y$ (normal equation) or gradient descent. Coefficient = change in y per unit of x, other features fixed.
- Assumptions (**LINE**): Linear relationship, Independent errors, Normal errors (for inference), Equal variance (homoscedasticity). Check residual plots, Q-Q plot.
- $R^2 = 1 - \text{SSres}/\text{SStot}$ (rises with every feature; use adjusted R^2 or validation error).
- **Multicollinearity:** correlated features give unstable coefficients; check VIF > 5-10; drop, combine, or use ridge.
- Outliers and leverage points pull the line; extrapolation is unsafe. Heteroscedasticity: log target, weighted least squares, robust SEs.

LOGISTIC REGRESSION

- $p = 1/(1 + e^{-z})$, $z = w \cdot x + b$. It models **log-odds** linearly: $\log(p/(1-p)) = z$. $\exp(w) = \text{odds ratio}$ per unit.
- Loss: log loss = $-[y \log p + (1-y) \log(1-p)]$, trained by maximum likelihood. Linear decision boundary. Multiclass: softmax.
- 0.5 is a default threshold, not a law: choose from the cost of errors (precision-recall curve).
- Regularised by default in sklearn ($C = 1/\lambda$). Probabilities are usually well calibrated; scale features for faster convergence.
- Good baseline: interpretable, fast, stable; weak on interactions unless you add features.

DECISION TREES

- Greedy recursive splits: choose the feature and threshold with the largest impurity reduction. **Gini** = $1 - \sum p^2$; **entropy** = $-\sum p \log_2 p$; regression uses variance.
- Example: node 50/50 has Gini 0.5; a split into 45/5 and 5/45 gives 0.18 each, gain 0.32.
- Controls: max_depth, min_samples_leaf, ccp_alpha (pruning). Deep trees overfit; small data changes alter the tree.
- No scaling needed, handles mixed types, captures interactions; step-like, poor at extrapolation.

RANDOM FOREST (BAGGING)

- Many trees on **bootstrap samples**, each split on a random feature subset ($\text{max_features} \sim \sqrt{p}$); average (regression) or vote. Variance falls; bias unchanged.
- **OOB score:** each tree is tested on rows it did not see (~37%): a free validation estimate.
- Importance: impurity-based favours high-cardinality features; prefer **permutation importance** or SHAP.
- Few hyperparameters ($n_estimators$, max_depth, min_samples_leaf); robust; cannot extrapolate beyond training range; slow, large models.

GRADIENT BOOSTING

- Trees built in **sequence**: each fits the residuals (negative gradient of the loss) of the ensemble so far; prediction = $\sum \text{learning_rate} \times \text{tree}$. Reduces **bias**; shallow trees (depth 3-8).
- Key knobs: learning_rate (small + many trees), n_estimators, max_depth, subsample, colsample_bytree, min_child_weight, L1/L2 (reg_alpha/lambda). Use **early stopping** on a validation set.
- **XGBoost:** regularised, second-order gradients, handles missing values, parallel. **LightGBM:** histogram bins, leaf-wise growth, fast on big data. **CatBoost:** native categorical handling, ordered boosting (less leakage).
- Usually the strongest model on tabular data. Overfits noisy labels; needs tuning; less interpretable (use SHAP).

ALGORITHM CARDS

Model	Idea	Scale?	Watch out
SVM	maximum-margin hyperplane; support vectors only; kernels (RBF) for curves	yes	C large = overfit, gamma large = wiggly; slow on big n
KNN	vote of k nearest points; no training	yes	slow predict, curse of dimensionality; choose k by CV
Naive Bayes	P(class) × product of P(feature given class) assuming conditional independence	no	poor probabilities; Laplace smoothing; great text baseline
Linear / logistic	weighted sum	yes	non-linearity needs features
Tree / forest / boost	thresholds	no	trees extrapolate poorly

- SVM margin = $2/|w|$; soft margin trades errors for width. KNN example: unscaled income (dollars) swamps age in distance, so scale first.
- Naive Bayes example: $P(\text{spam} \mid \text{"free"}) = 0.4 \times 0.5 / (0.4 \times 0.5 + 0.6 \times 0.05) = 87\%$.

ENSEMBLING

- Bagging**: parallel models on bootstrap samples, average: cuts variance (random forest).
- Boosting**: sequential error correction: cuts bias (GBM, AdaBoost).
- Stacking**: a meta-model learns from base models' **out-of-fold** predictions (in-sample predictions leak).
- Voting/averaging**: combine diverse models. Gains need diversity; costs latency and explainability.
- Pick an algorithm**: start with a simple baseline (logistic/linear) → gradient boosting for tabular → linear when explanation or regulation matters → KNN/SVM for small data → deep learning for images, text, audio.

OPTIMISATION

- Gradient descent: $w \leftarrow w - \eta \text{grad } L(w)$. Too large η diverges (NaN loss); too small is slow. Scale features.

- Batch** (all data), **stochastic** (one row, noisy), **mini-batch** (32-512, the default). **Momentum** smooths steps; **Adam** keeps per-parameter adaptive rates (default for deep nets); **RMSprop**, **AdaGrad**.
- Convex losses (linear, logistic) have one global minimum; neural nets have many local ones and saddle points. Use learning-rate schedules and early stopping.
- Normal equation $O(p^3)$ vs GD $O(n \cdot p)$ per step: GD for many features.

CROSS-VALIDATION AND TUNING

- k-fold** (k = 5): train on k - 1 folds, validate on one, average. **Stratified** for classes, **group k-fold** for entities, **TimeSeriesSplit** (forward chaining) for time.
- Grid search** (all combinations), **random search** (more efficient), **Bayesian/Optuna** (learns from trials). Tune on validation folds, test set once. **Nested CV** for an honest estimate when tuning heavily.

```
from sklearn.pipeline import
make_pipeline
from sklearn.preprocessing import
StandardScaler
from sklearn.linear_model import
LogisticRegression as LR
from sklearn.model_selection import
GridSearchCV
pipe = make_pipeline(StandardScaler(),
LR())
gs = GridSearchCV(pipe,
{"logisticregression__C": [.1, 1,
10]}),
cv=5, scoring="roc_auc")
gs.fit(X_train, y_train)
gs.best_params_
```

Scaler inside the pipeline = refit per fold = no leakage.

TYPICAL HYPERPARAMETERS (SKLEARN)

Model	Tune first	Typical range
Ridge / Lasso	alpha	1e-4 to 1e2, log scale
LogisticRegression	C (1/λ), penalty	1e-3 to 1e2
RandomForest	n_estimators, max_depth, min_samples_leaf, max_features	200-500, 5-30, 1-20, sqrt
GradientBoosting / XGB	learning_rate, n_estimators, max_depth, subsample	.01-.3, 100-2000, 3-8, 1e-4
SVM (RBF)	C, gamma	1e-2 to 1e3, to 1
KNN	n_neighbors, weights	3-30, uniform/distance
DecisionTree	max_depth, min_samples_leaf, ccp_alpha	3-15

INTERPRETING MODELS

- Linear/logistic**: coefficients (scaled features) and odds ratios; check significance with statsmodels.
- Trees/forests**: impurity importance (biased); **permutation importance** (shuffle a feature, measure the score drop) is better.
- SHAP**: per-prediction additive contributions; global bar and beeswarm plots. **PDP/ICE**: average effect of a feature. **LIME**: local linear surrogate.
- Calibration: reliability curve; fix with Platt scaling or isotonic regression (CalibratedClassifierCV). Needed when probabilities drive decisions.
- Correlated features share importance; interpretation is not causation.

EVERYDAY SKLEARN

```
from sklearn.ensemble import (
RandomForestClassifier as RF)
m = RF(n_estimators=300, random_state=0)
m.fit(X_tr, y_tr)
p = m.predict_proba(X_te)[: , 1]
from sklearn.model_selection import (
cross_val_score)
cross_val_score(m, X, y, cv=5,
scoring="f1")
from sklearn.metrics import roc_auc_score
roc_auc_score(y_te, p)
```

fit on train only; transform test with train-fitted objects; set random_state.

WHEN IT BREAKS: SYMPTOM -> TRY

Symptom	Try
Overfits	regularise, prune, more data, bagging, early stop
Underfits	features, interactions, bigger model, lower λ
Slow training	subsample, LightGBM, simpler model, fewer features
Unbalanced	class weights, threshold, PR-AUC (see Module 8)
log Unstable	drop collinear features, ridge
coefficients	
Offline great, online poor	leakage, train-serving skew, drift, wrong metric

SAY IT IN 20 SECONDS

- Bias-variance**: simple models underfit, flexible ones memorise noise.
- Bagging vs boosting**: average independent models vs fix errors in sequence.
- L1 vs L2**: L1 zeroes weights, L2 shrinks them.
- Leakage**: using information from the future or the label.
- Idea, reason, example, trade-off.